

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Analytical Problem Solving

Course Code:	CSA0313(Slot D)	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	Bhavya S	Reg. No.:	192524376
Section / Batch:	2 nd Year	Date:	29.7.26

Code of Conduct

I Bhavya S(192524376)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Bhavya S

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure	List Operations	1	20
Search Data Structure	Binary Search	2	20
Stack Data Structure	Stack Operations	3	20
Arithmetic Expression Evaluations	Stack Notations	4	20
Queue Data Structures	Priority Queue	5	20
GRAND TOTAL:			100 Marks

Annexure A – Analytical Problem Solving

1. Show the linked list configuration resulting from each series of list operations using the List ADT. Assume that lists L1 and L2 are empty at the beginning of each series. Show where the current position is in the list. (a) L1.append (10); L1.append (20); L1.append (15); (b) L2.append (10); L2.append (20); L2.append (15); L2.moveToStart (); L2.insert (39); L2.next (); L2.insert (12). (20 Marks)
2. Implement Binary searching Algorithm for searching an element and its position having the following 10 elements. Elements are 66, 111, 212, 323, 432, 543, 643, 649, 777, 888. Case 1: Key element = 111, Case 2: Key element=888, Case 3: Key element = 999. Analyse the concept in detail. (20 Marks)
3. Perform the following operations in LIFO data structure having an array size of 5. Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(), Pop(). Print the peak element of the data structure and its index position. With a neat sketch explained in detail. (20 Marks)
4. Convert the expression $((a + b) + c * (d + e) + f) * (g + h)$ to the following expressions
 - a. Prefix expression
 - b. Postfix expression. (20 Marks)
5. A queue-based ticketing system serves customers in the order of arrival, but VIP customers must be prioritized occasionally. Analyze how modifying a simple queue into a priority-based queue affects fairness, waiting time, and computational complexity. Compare both systems using analytical reasoning. (20 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

CO2-AT1

1. Problem Understanding:

The task is to perform the given list ADT operations on two initially empty linked list (L1 and L2). After each operation, the linked list configuration & the current position (cursor) must be shown.

Data Structure Selection:

Selected Data Structure: Single linked list (List ADT).

Justification:

- Supports efficient insertion & append operations.
- Dynamic memory Allocation.
- Cursor (current position) can be moved using `moveToStart()` and `next()`.

Algorithm Design:

1. Start with an empty linked list.
2. Execute each operation in the given order.
3. Update the current position after every operation.
4. Display the linked list after each step.

a) L1 Operations

Initially

$L1 = \text{NULL}$

Step 1: $L1.append(10)$

$[10]$

↓

current

Step 2: $L1.append(20)$

$[10] \rightarrow [20]$

↓

current

Step 3: $L1.append(15)$

$[10] \rightarrow [20] \rightarrow [15]$

↓

current

Step 4: final $L1$:

$[10] \rightarrow [20] \rightarrow [15]$

b) L2 Operations:

Initially

$L2 = \text{NULL}$

Step 1: $L2.append(10)$

$[10]$

↓

current

Step 2: $L2.append(20)$

$[10] \rightarrow [20]$

↓

current

Step 3: $L2.append(15)$

$[10] \rightarrow [20] \rightarrow [15]$

↓

current

Step 4: moveToStart()

$[10] \rightarrow [20] \rightarrow [15]$
↓
Current

Step 5: Insert(39)

$[39] \rightarrow [10] \rightarrow [20] \rightarrow [15]$
↓
Current

Step 6: next()

$[39] \rightarrow [10] \rightarrow [20] \rightarrow [15]$
↓
Current

Step 7: Insert(12)

$[39] \rightarrow [12] \rightarrow [10] \rightarrow [20] \rightarrow [15]$
↓
Current

Final L2:

$[39] \rightarrow [12] \rightarrow [10] \rightarrow [20] \rightarrow [15]$

Analysis:

- append inserts elements at the end of the list.
- moveToStart() moves the current pointer to the beginning.
- next() moves the current pointer one node forward.
- insert(x) inserts the new element at the current position.
- Linked lists allow efficient insertion without shifting elements.

2. Problem Understanding:

The given array is already sorted in ascending order. Binary Search repeatedly divides the search space into 2 halves until the key is found or the search interval becomes empty.

Data Structure Selection:

Data Structure: Sorted array

Justification:

- Binary Search works only on sorted data.
- Provides efficient searching.
- Time Complexity = $O(\log n)$.

Algorithm Design

1. Set $low = 0$, $high = n - 1$.
2. Find middle element:

$$mid = \left(\frac{low + high}{2} \right)$$

3. If $key == array[mid]$ return position.
4. If $key < array[mid]$, search left half.
5. If $key > array[mid]$, search right half.
6. Repeat until key is found or $low > high$.

Array:

Index	0	1	2	3	4	5	6
Element	66	111	212	323	432	543	643

Case 1: Key = 111

Iteration	Low	High	Mid	Value	Result
1	0	9	4	432	Search Left
2	0	3	1	111	Found

Position = Index 1 (2nd Position)

Case 2: Key = 888

Iteration	Low	High	Mid	Value	Result
1	0	9	4	432	Search Right
2	5	9	7	649	Search Right
3	8	9	8	777	Search Right
4	9	9	9	888	Found

Position = Index 9 (10th Position)

Case 3: Key = 999

Iteration	Low	High	Mid	Value	Result
1	0	9	4	432	Search Right
2	5	9	7	649	Search Right
3	8	9	8	777	Search Right
4	9	9	9	888	Search Right
5	10	9	-	-	Not Found

Analysis:

→ Binary Search reduces the search space by half in each iteration.

→ Best Case: $O(1)$

→ Average Case: $O(\log n)$

→ Worst Case: $O(\log n)$

→ Space Complexity: $O(1)$

3. Problem Understanding:

The problem requires performing stack (LIFO) operations on an array of size 5. After each Push & Pop operation, the stack must be updated & finally the top element & its index position should be identified.

Data Structure Selection:

Data Structure: Stack (Array Implementation)

Justification:

- Stack follows the last in first out (LIFO) principle.
- Push inserts an element at the top.
- Pop removes the topmost element.
- Both operations take $O(1)$ time.

Algorithm Design:

1. Create an empty stack of size 5.
2. Perform Push operations until the stack is full.
3. Perform Pop operations to remove the top element.
4. Repeat the given sequence.
5. Display the final stack.
6. Print the top element & its index.

Stack Size = 5

Operation	Stack (Bottom - Top)	Top Index
Push (45)	45	0
Push (22)	45, 22	1
Push (11)	45, 22, 11	2
Push (77)	45, 22, 11, 77	3
Pop()	45, 22, 11	2
Push (44)	45, 22, 11, 44	3
Push (33)	45, 22, 11, 44, 33	4
Pop()	45, 22, 11, 44	3
Pop()	45, 22, 11	2
Pop()	45, 22	1

Final Stack:

22
45

Top (Peak) Element = 22
Index Position = 1

Analysis:

- Push inserts an element at the top of the stack.
- Pop removes the most recently inserted element.
- The stack follows the LIFO (Last in First out) principle.
- Push & Pop operations require $O(1)$ time complexity.

4. Problem Understanding:

The given infix expression must be converted into prefix & postfix notations using stack-based expression conversion. Prefix places the operator before its operands, while postfix places the operator after this operand.

Data Structure Selection:

Data Structure: Stack

Justification:

- Stack follows the LIFO principle
- Efficiently handles operators & parentheses during expression conversion.
- Time Complexity = $O(n)$

Algorithm Design:

1. Scan the infix expression.
2. Push operators & parentheses onto the stack.
3. Pop operators according to precedence rules.
4. Generate Prefix & Postfix expressions.
5. Display the converted expressions.

Given Infix Expression:

$$((a+b)+c*(d+e)+f)*(g+h)$$

Step	Expression Process	Prefix Expression	Postfix Expression
1.	a	a	a
2.	b	b	b
3.	(a + b)	+ ab	ab +
4.	d	d	d
5.	e	e	e
6.	(d + e)	+ de	de +
7.	c	c	c
8.	c * (d + e)	* c + de	c de + *
9.	(a + b) + c * (d + e)	+ + ab * c + de	ab + c de + * +
10.	f	f	f
11.	((a + b) + c * (d + e) + f)	+ + + ab * c + de f	ab + c de + * + f +
12.	g	g	g
13.	h	h	h
14.	(g + h)	+ gh	gh +
15.	((a + b) + c * (d + e) + f) * (g + h)	* + + + ab * c + de f + gh	ab + c de + * + f + gh + *

Analysis :

- Prefix notation doesn't require parentheses
- Postfix notation also eliminate parentheses

Result :

The given infix expression was successfully converted into:

Prefix: * + + + ab * c + de f + gh

Postfix: ab + c de + * + f + gh + *

5. Problem Understanding:

The problem compares a Simple Queue (FIFO) with a Priority Queue in a ticketing system. The aim is to analyze how giving priority to VIP customers affects service order, fairness, waiting time & computational complexity.

Data Structure Selection:

Data Structure: Queue

- **Simple Queue (FIFO)**: Customers are served in the order they arrive.
- **Priority Queue**: Customers with higher priority (VIP) are served before normal customers.

Justification:

- Queue is suitable for ticket booking systems.
- Priority Queue efficiently handles urgent or VIP customers.

Algorithm Design:

Simple Queue (FIFO):

1. Customer enters the queue.
2. Customer is added at the rear.
3. Customer at the front is served first.
4. Repeat until the queue becomes empty.

Priority Queue:

1. Customer enters the queue.
2. Check customer priority (VIP or Normal).
3. Insert according to priority.
4. Serve the highest-priority customer first.
5. Repeat until the queue becomes empty.

Comparison Table:

Criteria	Simple Queue (FIFO)	Priority Queue
Principle	First in First Out	Highest Priority first.
Service Order	Arrival Order	VIP served before normal customers.
Fairness	High	Moderate (Normal customers may wait longer).
Waiting Time	Same for everyone	Reduce for VIP, increased for normal customers.
Complexity (Insertion)	$O(1)$	$O(\log n)$ (Heap Implementation)
Complexity (Deletion)	$O(1)$	$O(\log n)$
Best Use	Banks, Billing Centers, Ticket Counters.	Hospitals, Emergency Services, VIP Ticket Booking

Analysis:

Simple Queue:

Advantages:

- Fair Service based on arrival.
- Easy to implement.
- Constant - time insertion & deletion ($O(1)$).

Disadvantages:

- Cannot handle urgent or VIP customers.
- Emergency request must wait.

Priority Queue:

Advantages:

- VIP or emergency customers receive faster service.
- Reduces waiting time for high - priority customers.
- Suitable for real - time applications.

Disadvantages:

- Normal Customers may experience longer waiting times.
- More Complex implementation.
- Higher Computational complexity than a simple queue.

Result:

- A Simple Queue provides fair service by following the FIFO principle, making it suitable for normal ticketing systems.
- A Priority Queue improves service for VIP customers by serving them 1st, reducing their waiting time but decreasing fairness for regular customers.
- Therefore, the choice depends on whether fairness or priority-based service is more important in the application.